Section 3: Feasibility Filter and Baseline Scheduling Policies

AMS 560 · Big Data Systems, Algorithms and Networks · Fall 2026 · Group 20 | Author: Jyotir

3.1 Overview

Our scheduler operates in two stages: a **feasibility filter** that removes nodes that physically cannot serve a request, followed by a **selection policy** that picks the best node from those that remain. This section describes both components.

The separation between filtering and selection is intentional. Hard physical constraints — node reachability, model availability, GPU memory, thermal limits, and power limits — are not preferences to be traded off; they are binary. A node whose KV cache is full will drop the request. A node above 85 °C is throttling. Routing to such a node wastes the request and degrades the SLO. By handling these constraints in a single, dedicated filter, every policy downstream is guaranteed to work only with nodes that can actually serve the request.

This design also enables controlled experimental comparison: every baseline policy and the learned policy (sliding-window LinUCB) receive exactly the same candidate set from the same filter. Any difference in measured goodput or adaptation time is therefore attributable to the selection logic, not to differences in which nodes each policy sees.

3.2 Feasibility Filter

3.2.1 Design

The filter takes a request and the current telemetry snapshot of all nodes and returns two lists: **allowed** (nodes that pass every rule) and **rejected** (nodes removed, each with a reason string). If **allowed** is empty, the router rejects the request entirely with reason `no_feasible_node`.

All thresholds are read from `feasibility_filter/configs/feasibility.yaml` at startup. No threshold is hardcoded in Python, allowing the operator to tune limits without touching the implementation.

```
temperature_limit_c: 85.0 power_headroom_min_kw: 2.0 kv_cache_max_frac: 0.90
allow_warming_nodes: true
```

3.2.2 The Five Rules

The filter evaluates each node against five rules in order. Evaluation stops at the first failure (fail-fast), and the reason string for that rule is recorded.

> **Rule 1 — Node Status.** The node's `status` field must be `"up"` or `"warming"`. Status is normalised to lowercase, so `"DOWN"`, `"Offline"`, and `"draining"` are all treated as `node_down`. A missing

`status` field is also treated as `node_down`. Nodes with status `"warming"` pass this rule and continue through all remaining rules without fast-pathing.

Rule 2 — Model Available. The model requested by the client must appear in the node's `models_loaded` list. A node that does not carry the required model returns reason `model_missing`. This matters in a geo-distributed cluster where different nodes may be configured for different model variants.

Rule 3 — KV Cache Memory. The filter checks whether the incoming request's tokens fit in the node's remaining KV cache. When `kv_capacity_tokens` is available, the check is token-exact: reject if *kv_used_tokens* + *tokens_needed* > *kv_capacity_tokens*. Otherwise the filter falls back to a fraction threshold: reject if `kv_used_frac ≥ 0.90`. In either case, rejection reason is `memory_full`. The token-exact check is strictly tighter: a node at 50% fill with 1,000-token capacity will correctly reject a 1,050-token request, whereas the fraction fallback alone would pass it.

Rule 4 — Power Headroom. The node must have at least 2.0 kW of available power above its current draw. A node below this threshold is at risk of hitting its power cap mid-request, causing thermal throttling and sudden latency spikes. Rejection reason: `power_limit`.

Rule 5 — Temperature. The node's GPU temperature must be strictly below 85 °C. A node at or above this threshold is throttling; further requests will see degraded throughput until the node cools. Rejection reason: `temperature_high`.

3.2.3 Warming Node Handling

A node in `"warming"` status has recently restarted or returned from an outage (Scenario S6). It is functionally available but may be slower than steady state while its caches and JIT compilation warm up.

If `allow_warming_nodes` is `true` (the default), a node that passes all five rules with status `"warming"` is admitted to the allowed list with the extra flag `_warming=True`. Downstream policies use this flag to apply a soft preference penalty — they still consider the node but prefer a fully warm node when one is available.

A key design point: warming nodes are **not** exempted from rules 2–5. A warming node that is also out of memory, over temperature, or missing the required model is rejected just as any other node would be. The `_warming` tag is a scheduling hint, not a safety override.

3.2.4 Rule Reference Table

Rule	Field Checked	Threshold	Rejection Reason
1 Status	status	must be "up" or "warming"	node_down
2 Model	models_loaded	requested model present	model_missing
3 Memory	kv_used_frac kv_capacity_tokens	token fit OR frac < 0.90	memory_full
4 Power	power_headroom_kw	≥ 2.0 kW	power_limit
5 Temperature	temperature_c	< 85 °C	temperature_high

3.3 Baseline Scheduling Policies

Baseline policies are simple, stateless (or nearly stateless) scheduling rules. Their purpose in this work is threefold: (1) a **lower bound on performance** — a learned policy that does not beat the best baseline does not justify its added complexity; (2) **ablation anchors** — each baseline isolates one signal (latency, load, energy cost) so we can understand which signals matter most in which scenarios; and (3) **stress-testing** — some baselines perform well in specific scenarios by construction, setting a ceiling for how much improvement is possible.

All five (and the optional sixth) baseline policies accept the `allowed` list from the feasibility filter and return a single `node_id`. They are implemented as subclasses of the common `Policy` interface so they are interchangeable with the learned policy.

3.3.1 Round-Robin (RR)

Maintains a per-instance counter. On each call, increments the counter and selects `candidates[counter % len(candidates)]`. Makes no assumptions about node state and distributes requests evenly across whatever candidates the filter passes.

Why it exists. Round-robin is the canonical load-distribution baseline. It is both simple and well-understood, making it a reliable lower bound.

Expected behavior across the six change scenarios:

Scenario	Expected behavior
S1 Stable	Spreads load evenly. Goodput near the cluster mean.
S2 Traffic burst	Distributes burst across all feasible nodes. May cause imbalance if one region fills faster and gets filtered out fi
S3 Price shift	Indifferent to price. Energy cost rises needlessly when cheap nodes are available.
S4 Thermal throttle	Adapts automatically once the hot node crosses 85 °C and is removed by the filter.
S5 Network degradation	Continues routing to the high-latency node, increasing TTFT for those requests.
S6 Outage + recovery	Removes downed node via filter; re-includes it on recovery. Distributes evenly throughout.

Weakness. Makes no use of any signal. A fast H100 and a slow A100 should not receive equal traffic, but round-robin gives them the same share.

3.3.2 Minimum RTT (MinRTT)

For each candidate, looks up the network round-trip time to the requesting user's region from the node's `rtt_ms` map. Selects the candidate with the lowest RTT. Falls back to the average RTT across all regions if the user's region is not found.

Why it exists. Network delay is the dominant component of TTFT for short requests routed across regions. MinRTT provides the best case for SLO compliance when compute is not the bottleneck.

Expected behavior across the six change scenarios:

Scenario	Expected behavior
S1 Stable	Routes to the geographically nearest node. Low TTFT, strong SLO compliance.
S2 Traffic burst	Concentrates traffic on the lowest-RTT node until it fills and is filtered out; then shifts to the next closest.
S3 Price shift	Indifferent to price. Routes to cheap nodes only by coincidence.
S4 Thermal throttle	RTT is unaffected by thermal state. Continues routing to the overloaded node until the filter removes it at 85 °C
S5 Network degradation	Reacts immediately as the degraded node's RTT rises. This is the scenario MinRTT was designed for.
S6 Outage + recovery	On recovery, immediately routes to the returned node if it is geographically closest.

Weakness. Ignores queue depth and running requests. Concentrates requests on the nearest node even when it is saturated.

3.3.3 Least Load (LL)

Computes `queue_len + running` for each candidate as a proxy for current load. Selects the candidate with the lowest value. Tie-breaking is by order in the candidate list (stable and deterministic).

Why it exists. Queue length and running request count are the most direct indicator of whether a node will process the next request quickly. Under steady-state traffic, least-load approximates optimal load balancing without any knowledge of node speed.

Expected behavior across the six change scenarios:

Scenario	Expected behavior
S1 Stable	Keeps all nodes near equal utilisation. Good general-purpose performance.
S2 Traffic burst	Spreads the burst across all nodes. More resilient than MinRTT. Expected to be the best simple baseline in this
S3 Price shift	Indifferent to price.
S4 Thermal throttle	The overheated node is likely the busiest; LL may already deprioritise it before the filter removes it.
S5 Network degradation	Ignores RTT. May route to a high-latency node if it is less loaded.
S6 Outage + recovery	On recovery, the returning node has zero load and will receive a burst of traffic until the system rebalances.

Weakness. `queue_len + running` is a count of requests, not a weighted measure of their cost. A mix of short and long requests may look lighter than it is.

3.3.4 Lowest Energy Cost (LE)

Selects the candidate with the lowest `energy_price_usd_kwh`. Before sorting by price, the policy excludes any candidate where `queue_len + running ≥ capacity` (node is saturated). If all candidates are saturated, the sort applies to the full list rather than returning an error.

Why it exists. Electricity price varies by region and time of day. NYISO and CAISO publish real-time marginal prices that can change by an order of magnitude across hours. LE is the natural baseline for Scenario S3 (price/renewable shift).

Expected behavior across the six change scenarios:

Scenario	Expected behavior
S1 Stable	Routes to cheapest region. Operating cost minimised but latency and load ignored; traffic concentrates on one
S2 Traffic burst	May flood the cheapest node. The saturation check prevents routing to a fully saturated node, but queue builds
S3 Price shift	Immediately shifts routing when the cheapest region changes. Expected to outperform all other baselines here.
S4 Thermal throttle	Indifferent to thermal state until the filter removes the node at 85 °C.
S5 Network degradation	Indifferent to RTT.
S6 Outage + recovery	On recovery, routes to the returned node if it is the cheapest option.

Weakness. Ignores all signals except price. A cheap node may be cheap because it is a remote region with high RTT.

3.3.5 Static Weighted (SW)

Computes a composite score for each candidate by min-max normalising three signals across the current candidate set and combining them with fixed weights:

```
score = w_rtt × norm_rtt + w_load × norm_load + w_price × norm_price
```

where *norm_x* is min-max normalised over the candidates (0 = best, 1 = worst). Default weights: `w_rtt = 0.5, w_load = 0.3, w_price = 0.2`. The candidate with the **lowest** score wins. If all candidates share the same value for a signal, normalisation returns 0 for all, and that signal has no effect.

Why it exists. Static Weighted is the best single-policy heuristic that considers all three optimisation criteria simultaneously — latency (RTT), throughput (load), and operating cost (price). It is the natural upper bound for non-adaptive baselines.

Expected behavior across the six change scenarios:

Scenario	Expected behavior
S1 Stable	Generally strong: routes to nodes that are nearby, lightly loaded, and affordable. Expected to be the best-perfo
S2 Traffic burst	The load signal reacts to the burst; static weights adjust routing toward less-loaded nodes. Performance depen
S3 Price shift	Price signal present at weight 0.2. Shifts some traffic to the newly cheap region but RTT weight prevents full co
S4 Thermal throttle	Load signal partially compensates (hot node is usually busy). Full removal handled by filter at 85 °C.

S5 Network degradation	RTT signal reacts immediately. SW naturally deprioritises the degraded node as its normalised RTT rises.
S6 Outage + recovery	On recovery, the returning node has zero load and low normalised scores, and may receive a disproportionate

Weakness. Weights are static. The same `w_rtt = 0.5` applies whether the cluster is latency-bound (S5) or price-sensitive (S3). The optimal weight vector changes with the operating scenario — this is the gap that sliding-window LinUCB fills.

3.3.6 Power-of-Two Choices (Po2) — Optional Baseline

Samples two candidates uniformly at random (without replacement) and selects the one with lower `queue_len + running`. If only one candidate is available, returns it directly.

Why it exists. Power-of-Two is a well-known result from load balancing literature (Mitzenmacher, 2001): sampling just two options and taking the better one achieves 99% of the load balance of optimal random assignment, with no central state. It tests whether the benefit of full load information over a random two-sample is significant in this cluster. Because the sample is random, a high-load node may occasionally be selected if neither of the two sampled candidates is the least-loaded. In a small cluster (5–10 nodes), Po2 will sometimes choose the second-best node. The gap between LL and Po2 measures the value of observing full cluster state.

3.4 What Baselines Do Not Do

Each baseline has a structural blindspot that the learned policy is designed to address. None of the baselines update their behaviour based on observed outcomes — they do not track TTFT, do not penalise nodes that produced SLO misses, and do not shift weights in response to a change event.

Baseline	Blindspot
Round-Robin	All signals
Min RTT	Load, price, thermal state
Least Load	RTT, price, request heterogeneity
Lowest Energy	RTT, load (until saturation)
Static Weighted	Fixed weights cannot adapt to scenario changes
Power-of-Two	Stochastic: misses best node proportional to cluster size

3.5 Integration with the Filter

All baseline policies receive the output of the feasibility filter, not the raw node list. They never need to check status, model availability, memory, power, or temperature — those checks are already done. If the filter returns an empty allowed list, the request is rejected before any policy is called.

The only filter-aware behaviour in the policies is the `_warming=True` tag. The Static Weighted policy and LinUCB both check this flag and can apply a load penalty to warming nodes. The other four baselines treat warming nodes identically to fully up nodes.

3.6 Test Coverage

The edge-case test suite (`edge_case_tests/test_filter_edge_cases.py`) covers six categories with 27 tests total. All 27 tests pass under `pytest`.

Test class	Tests	What is verified
TestAllNodesTooHot	4	All 4 nodes rejected; empty allowed list does not crash; reasons are temperature_high
TestOnlyOneNodeFeasible	5	Correct node returned; 3 rejected with correct reasons; all 6 baselines handle single-ca
TestWarmingNode	5	Warming node admitted with _warming=True; healthy node not tagged; warming reject
TestWarmingNodeFailsSafetyRules	5	Warming nodes with wrong model, full memory, low power, or high temperature are stil
TestLooseStatusValues	5	"DOWN", "offline", "draining", missing status all rejected; "up" passes (verifies status no
TestTokenAwareMemoryCheck	3	Token-exact check rejects oversized request; passes if capacity allows; fraction fallbac

3.7 Summary

The feasibility filter and five baseline policies form Jyotir's contribution to the evaluation framework. The filter enforces hard physical constraints before any policy sees the candidate list, ensuring fair and safe comparison across all policies. The baselines span the space of simple one-signal and multi-signal heuristics, from fully stateless (Round-Robin) to partially adaptive (Static Weighted). Their performance across scenarios S1–S6 provides the lower bound against which the adaptive sliding-window LinUCB policy is evaluated in Sections 4 and 5.